CSC 4505 (2025/26)
Graphics and Visual Computing

Lecture 2: Fundamentals of Visual Computing

Lighton Phiri <lighton.phiri@unza.zm>
Department of Computing and Informatics
University of Zambia
https://datalab.unza.zm/people/lightonphiri

Lecture Series Outline

- The Visual Computing Spectrum
- Deconstruction (Top-Down)
- Geometry (The Input)
- The Coordinate Systems
- The Image (The Output)
- The Bridge

Lecture Series Outline

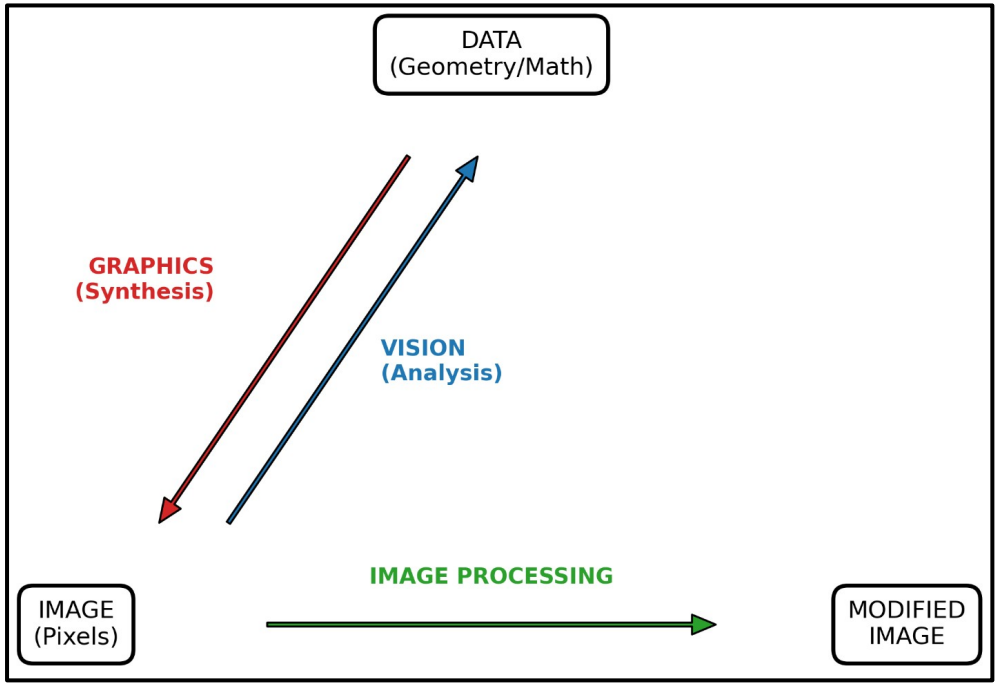
- **The Visual Computing Spectrum**
  - Synthesis vs. Analysis vs. Processing
- **Deconstruction (Top-Down)**
- **Geometry (The Input)**
- **The Coordinate Systems**
- **The Image (The Output)**
- **The Bridge**

The Spectrum: Synthesis vs. Analysis (1/2)

- **Visual Computing is the intersection of three fields defined by data flow**
  - Computer Graphics (Synthesis): Data -> Image
    - Building a model of the Kariba Dam
  - Computer Vision (Analysis): Image -> Data
    - Detecting ore grades from camera feeds
  - Image Processing: Image -> Image
    - Enhancing satellite photos

The Spectrum: Synthesis vs. Analysis (2/2)

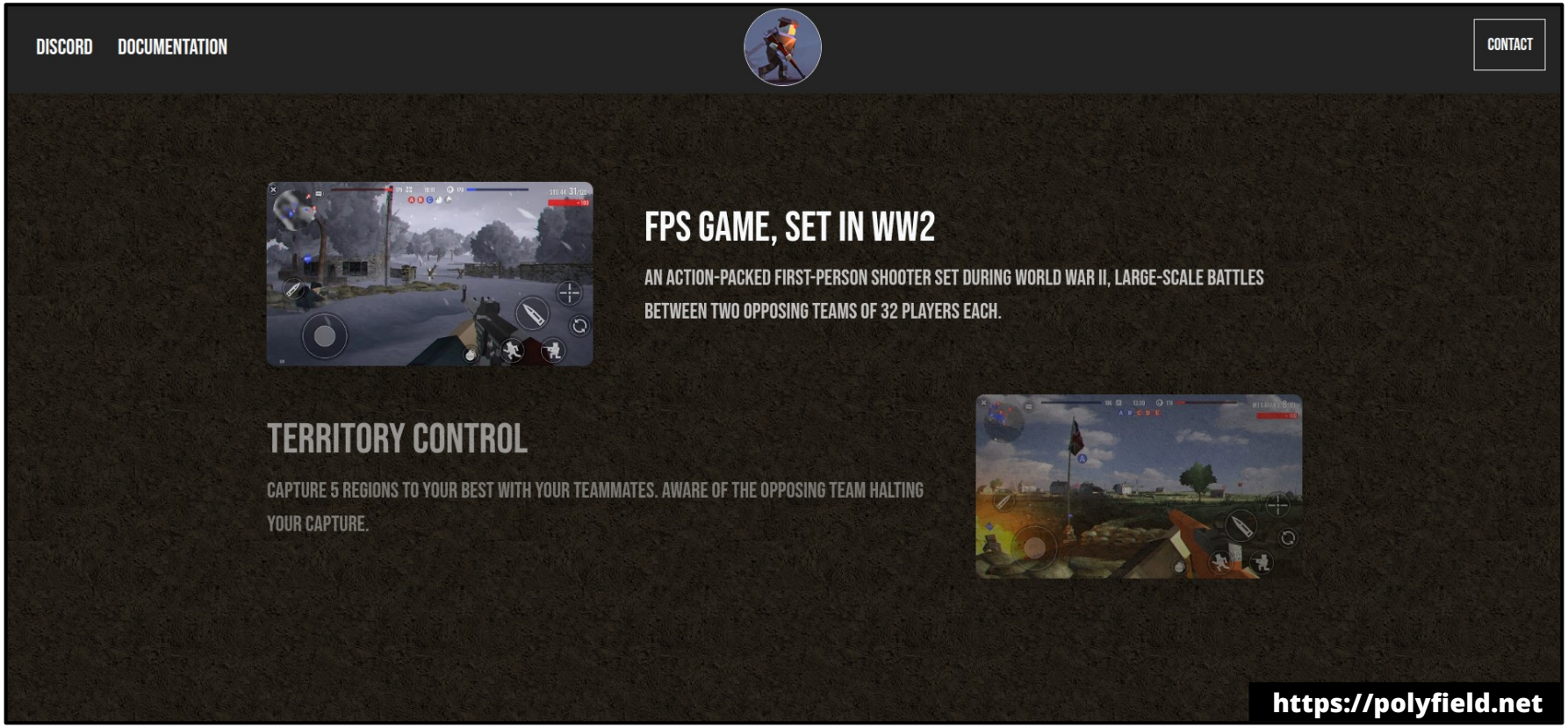
- **Computer Graphics (Synthesis)**
  - Data -> Image
- **Computer Vision (Analysis)**
  - Image -> Data
- **Image Processing**
  - Image -> Image



Lecture Series Outline

- **The Visual Computing Spectrum**
- **Deconstruction (Top-Down)**
  - Case Study: Polyfield
  - From Scene to Wireframe
- **Geometry (The Input)**
- **The Coordinate Systems**
- **The Image (The Output)**
- **The Bridge**

The Case Study: Polyfield (1/10)



The Case Study: Polyfield (2/10)

The Case Study: Polyfield (3/10)

- **Look at this scene from the game Polyfield**
  - Perception: We see a soldier running through grass; we see light, shadow, and atmosphere
  - The Reality: The screen is perfectly flat
    - The "World" is a data structure

The Case Study: Polyfield (4/10)

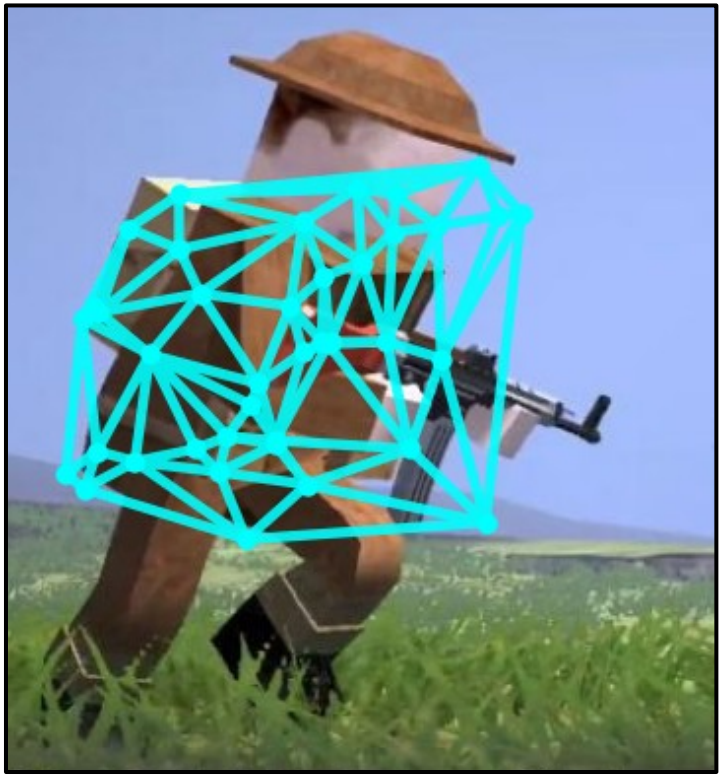
- **The Reality: The screen is flat**
  - The computer is performing a magic trick using three components:
    - Geometry: The shape of the objects
    - Texture: The "Skin" painted on top
    - Lighting: The brightness calculation

The Case Study: Polyfield (5/10)

- **Before we deconstruct the image, we must define what we are removing**
  - Texture (The Skin)
    - A 2D image (like a wallpaper) wrapped around a 3D object to give it detail (color, patterns)
    - In Polyfield: The camouflage pattern on the uniform
  - Lighting (The Physics)
    - Mathematical calculations that determine how bright a pixel is based on the angle of the "sun"
    - In Polyfield: The side of the soldier facing the sun is bright; the back is dark

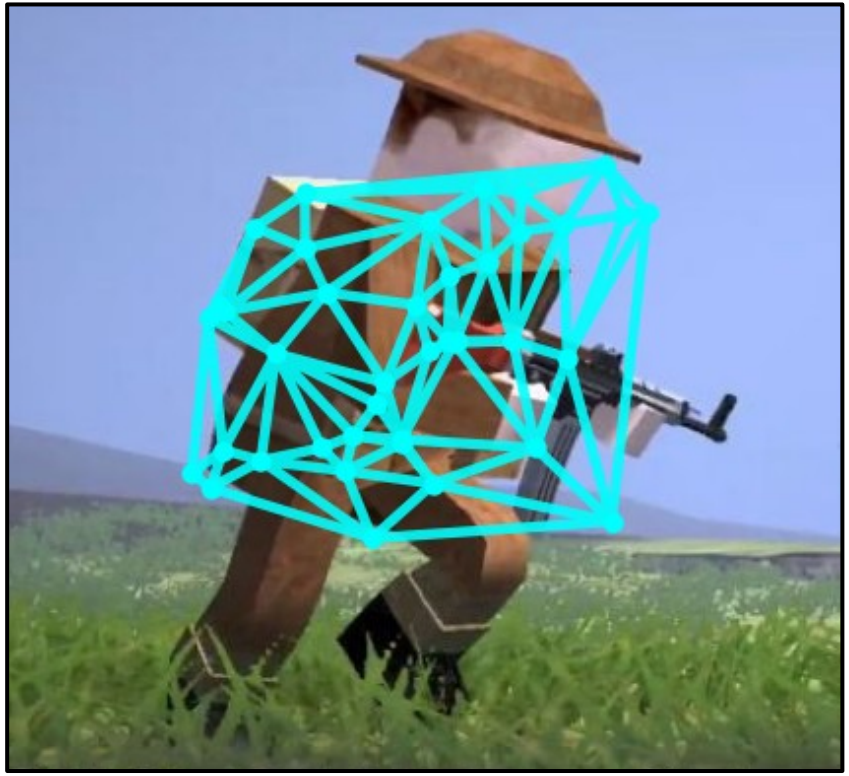
The Case Study: Polyfield (6/10)

- **If we turn off Textures and Lighting, we reveal the Geometry**
  - Polygon Mesh: A collection of vertices, edges, and faces that define the shape of a polyhedral object in 3D graphics
  - The soldier is not a solid volume. He is a hollow shell made of interconnected lines



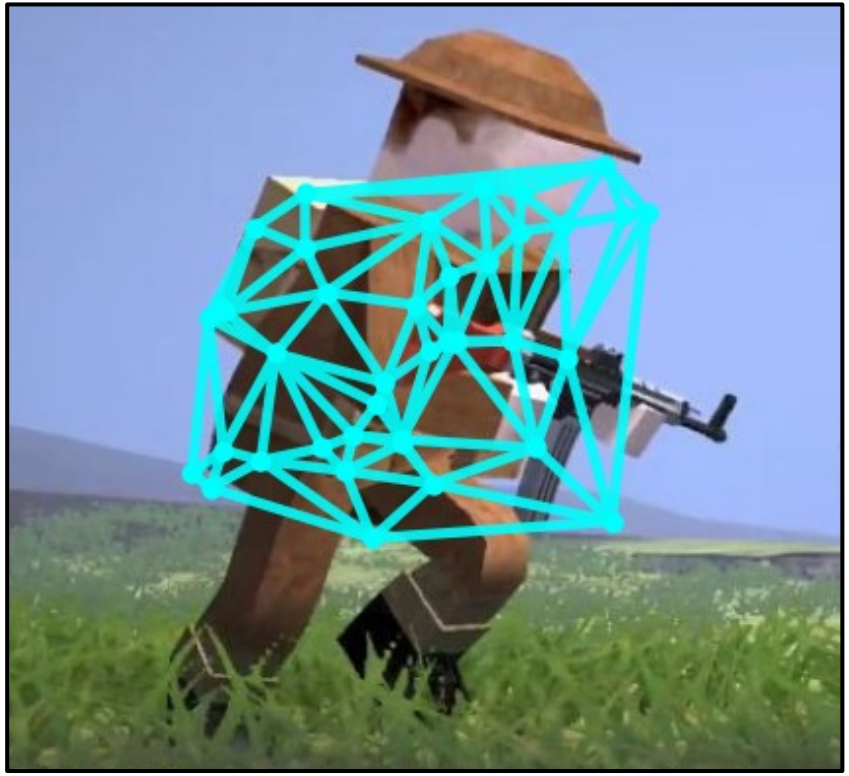
The Case Study: Polyfield (7/10)

- **If we turn off Textures and Lighting, we reveal the Geometry**
  - Notice how the curves are not real curves; they are straight lines connected at angles



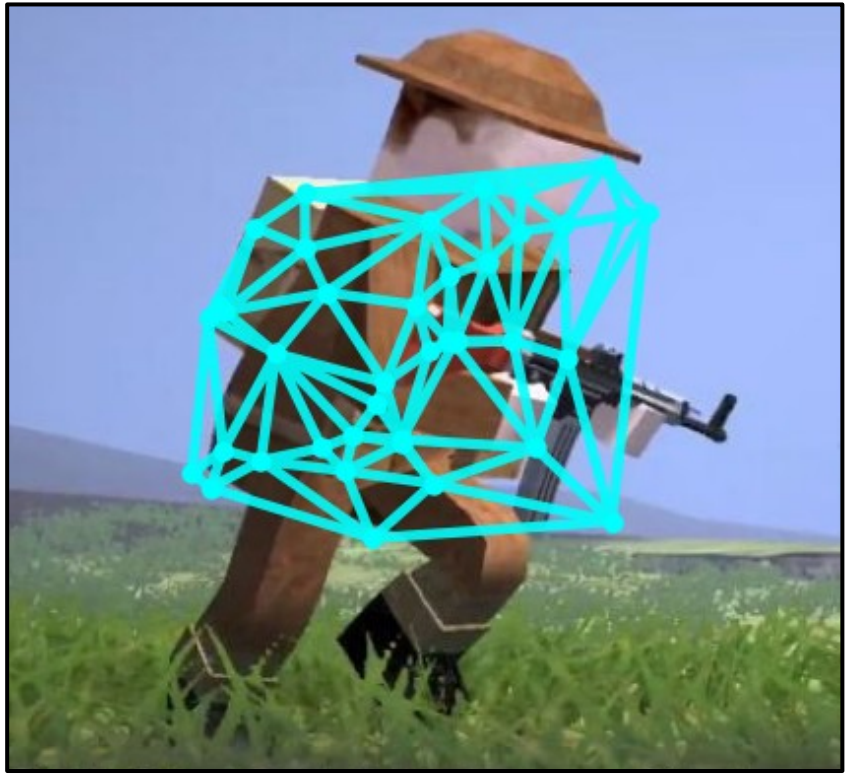
The Case Study: Polyfield (8/10)

- **The “Atomic Unit” is “The Triangle”**
- **Why is the Polyfield soldier made of triangles?**
  - 1. Planarity: 3 points always form a perfectly flat plane. (4 points can be warped)



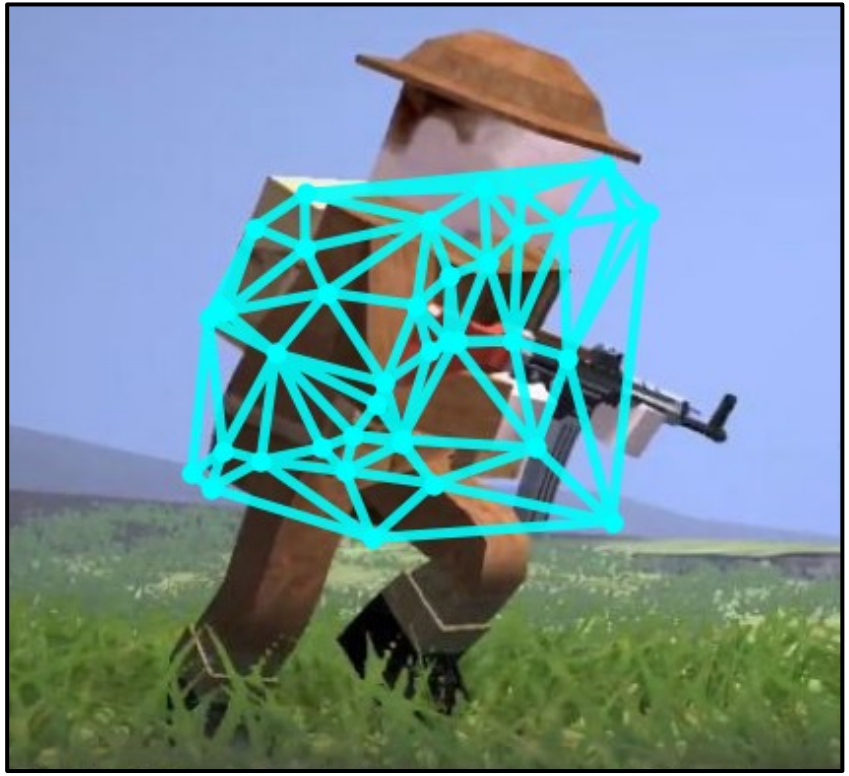
The Case Study: Polyfield (9/10)

- **The “Atomic Unit” is “The Triangle”**
- **Why is the Polyfield soldier made of triangles?**
  - 2. Simplicity: Any complex shape (a face, a terrain, a car) can be broken down into triangles



The Case Study: Polyfield (10/10)

- **The “Atomic Unit” is “The Triangle”**
- **Why is the Polyfield soldier made of triangles?**
  - 3. Hardware: The GPU is a factory designed to process triangles mathematically.

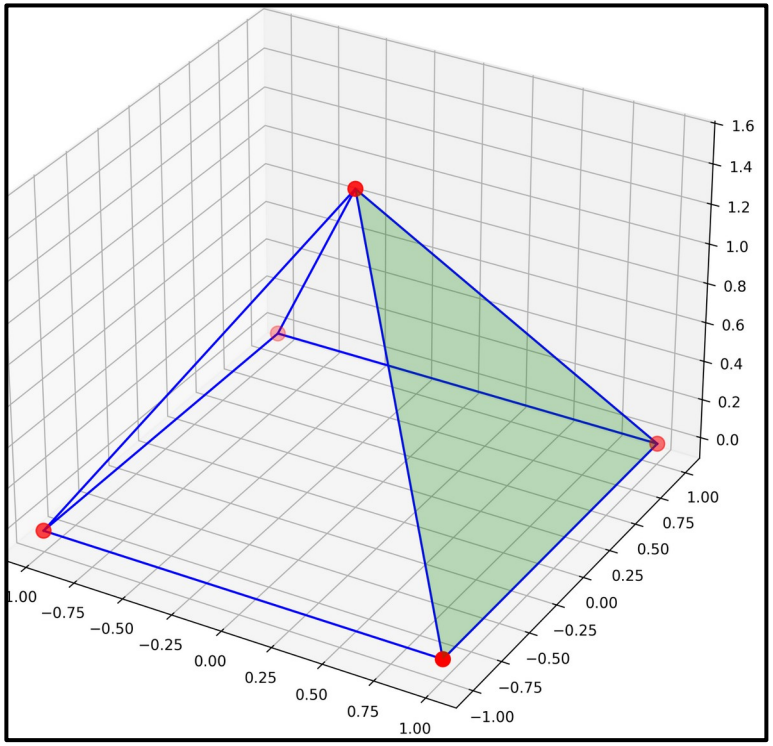


Lecture Series Outline

- **The Visual Computing Spectrum**
- **Deconstruction (Top-Down)**
- **Geometry (The Input)**
  - The Mesh: Vertices, Edges, Faces
  - Code: Defining shapes in NumPy
- **The Coordinate Systems**
- **The Image (The Output)**
- **The Bridge**

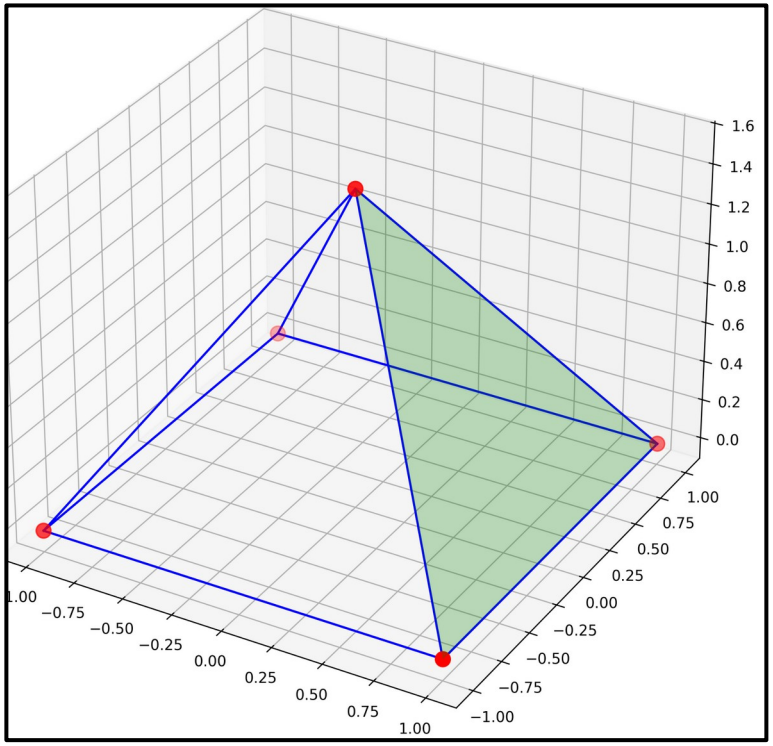
Geometry (The Input) (1/2)

- **The Mesh Hierarchy**
  - A 3D Object (Mesh) is not a single block of data
  - It is a hierarchy of three distinct elements
    - Vertex
    - Edge
    - Face



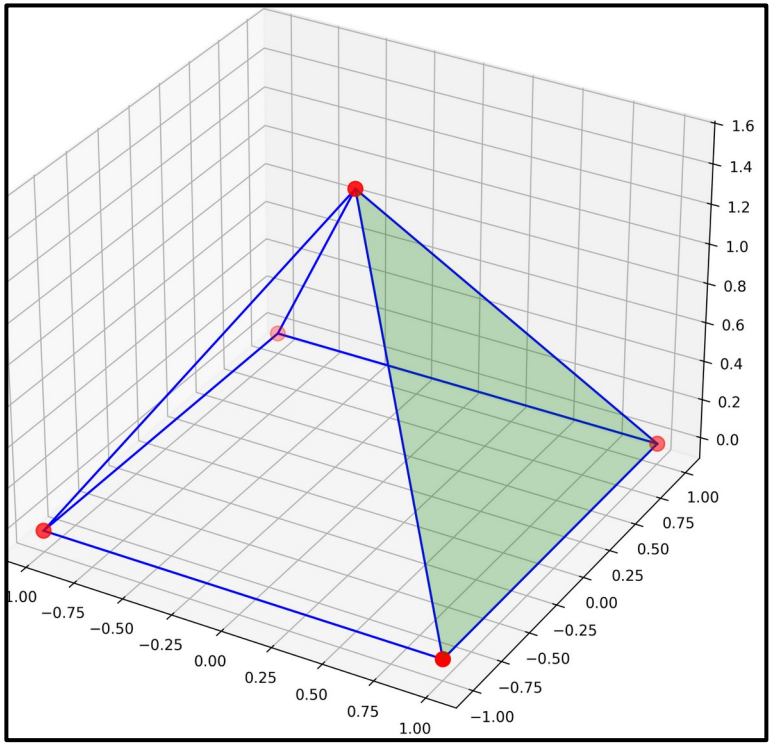
Geometry (The Input) (2/2)

- **The Mesh Hierarchy**
- **A 3D Object (Mesh) is a strict hierarchy of data**
  - 1. Vertex (0D)
    - A point in space (x, y, z)
  - 2. Edge (1D)
    - A line connecting two vertices
  - 3. Face (2D)
    - A surface enclosed by 3 edges (The Triangle)



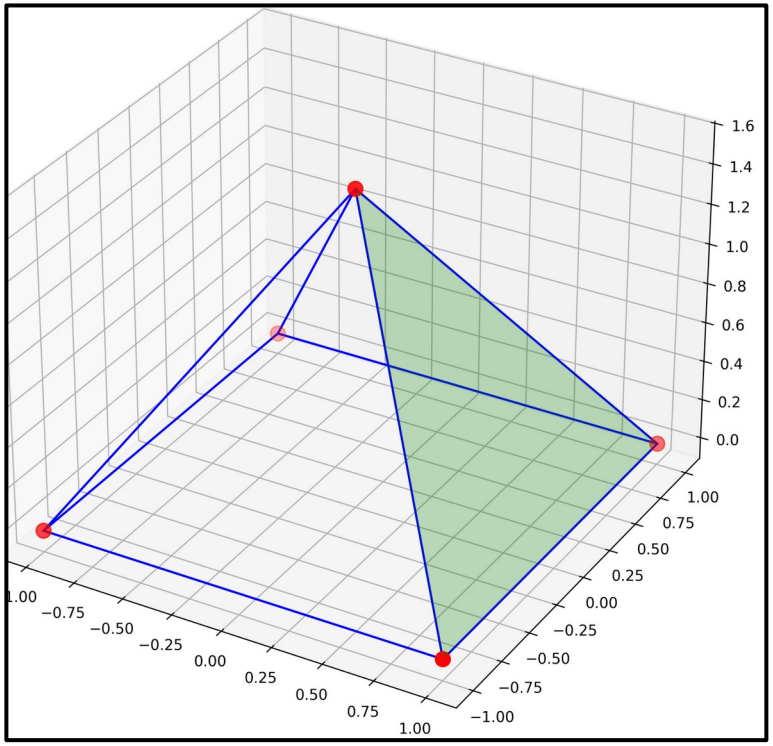
Geometry: The Vertex (1/7)

- **A Vertex is the fundamental unit of 3D data**
  - It is NOT just a position. It is a Data Container
  - Standard Vertex Attributes
    - 1. Position: (x, y, z)
      - Where in space is the point located?
    - 2. Color: (r, g, b)
      - What is the color of the vertex?



Geometry: The Vertex (2/7)

- **A Vertex is the fundamental unit of 3D data**
  - It is NOT just a position. It is a Data Container
  - Standard Vertex Attributes
    - 3. Normal: (nx, ny, nz)
      - Which way is it facing? (For lighting)
    - 4. Texture Coordinate: (u, v)
      - How do we wrap an image on it?



Geometry: The Vertex (3/7)

- **How do we represent a Triangle in Python?**
  - We can use NumPy Arrays because they are contiguous in memory (fast for GPU)

```
# A Triangle Mesh (3 Vertices)
# Format: [x, y, z]
var_vertex_data = np.array([
    [-1.0, -1.0,  0.0], # Vertex 1
    [ 1.0, -1.0,  0.0], # Vertex 2
    [ 0.0,  1.0,  0.0]  # Vertex 3
], dtype=np.float32)
```

Geometry: The Vertex (4/7)

- **How do we represent a Triangle in Python?**
  - We can use NumPy Arrays because they are contiguous in memory (fast for GPU)

```
(venv-csc4505_graphics) lightonphiri@ThinkPad-T14:~/Projects/work/2026/misc-unza26-csc4505$ p
ython
Python 3.12.3 (main, Jan  8 2026, 11:30:50) [GCC 13.3.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> import numpy as np
>>> var_vertex_np_data = np.array([[-1.0, -1.0, 0.0], [1.0, -1.0, 0.0], [0.0, 1.0, 0.0]])
>>> var_vertex_np_data
array([[-1., -1.,  0.],
       [ 1., -1.,  0.],
       [ 0.,  1.,  0.]])
>>> var_vertex_np_data.ndim
2
>>>
```

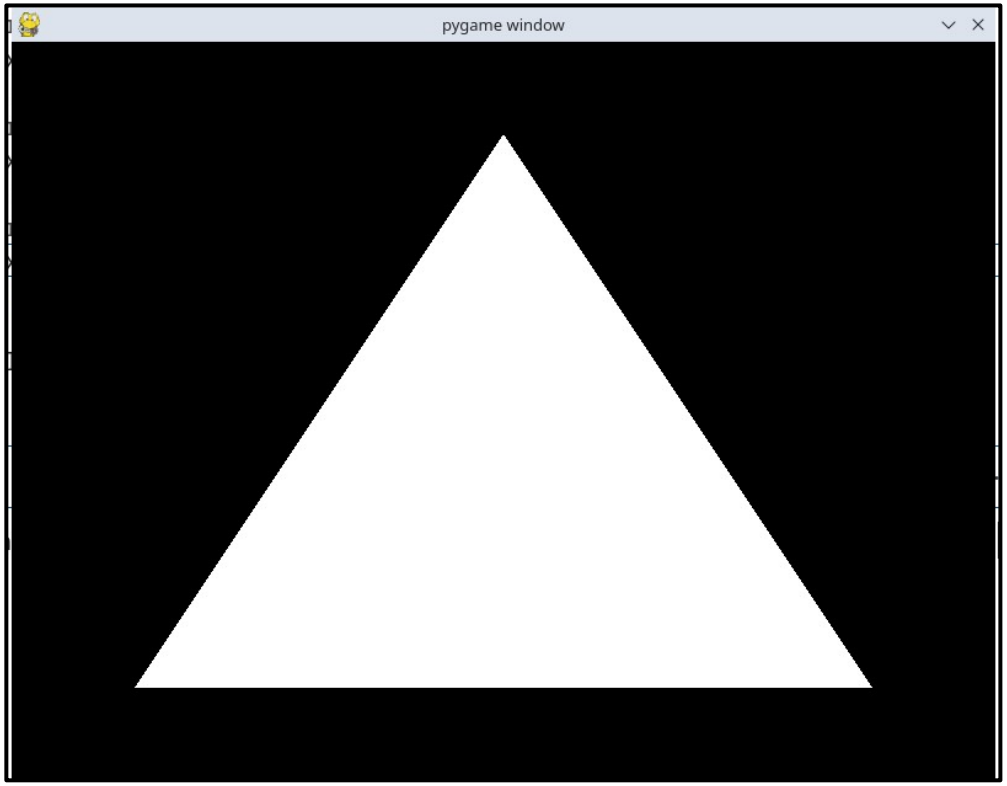
Geometry: The Vertex (5/7)

- We define the data in Python, but we must send it to the GPU to draw it

```
# PyOpenGL Syntax (Legacy Mode for demonstration)
glBegin(GL_TRIANGLES)
    glVertex3f(-1.0, -1.0, 0.0)
    glVertex3f( 1.0, -1.0, 0.0)
    glVertex3f( 0.0,  1.0, 0.0)
glEnd()
```

Geometry: The Vertex (6/7)

```
# PyOpenGL Syntax (Legacy Mode for demonstration)
glBegin(GL_TRIANGLES)
    glVertex3f(-1.0, -1.0, 0.0)
    glVertex3f( 1.0, -1.0, 0.0)
    glVertex3f( 0.0,  1.0, 0.0)
glEnd()
```



Geometry: The Edge (1/2)

- **A Vertex is a dot. An Edge is the relationship between two dots**
  - A line segment connecting two specific vertices (V_0 and V_1)
  - Mathematical Function:Linear Interpolation
    - Any point P on the edge is: P = (1-t)V_0 + (t)V_1
  - Why it matters
    - Wireframes: Edges define the "skeleton
    - Gradients: If V_0 is Red and V_1 is Blue, the Edge calculates the gradient (Purple) between them

Geometry: The Edge (2/2)

- **How do we tell the computer "These two points are connected"?**
  - We change the drawing mode to GL_LINES

```
glBegin(GL_LINES)
    # Start Point
    glVertex3f(-1.0, 0.0, 0.0)
    # End Point
    glVertex3f( 1.0, 0.0, 0.0)
glEnd()
```

Geometry: The Face

- **Vertices and Edges form the skeleton. The Face is the skin.**
  - A closed surface formed by connecting 3 or more vertices
  - The Triangle Rule
    - We rarely use Squares (Quads) in game engines
    - Why?
      - 3 points are Always Planar
      - 4 points can be "bent" (non-planar), which breaks math calculations
  - Polyfield uses Triangles exclusively

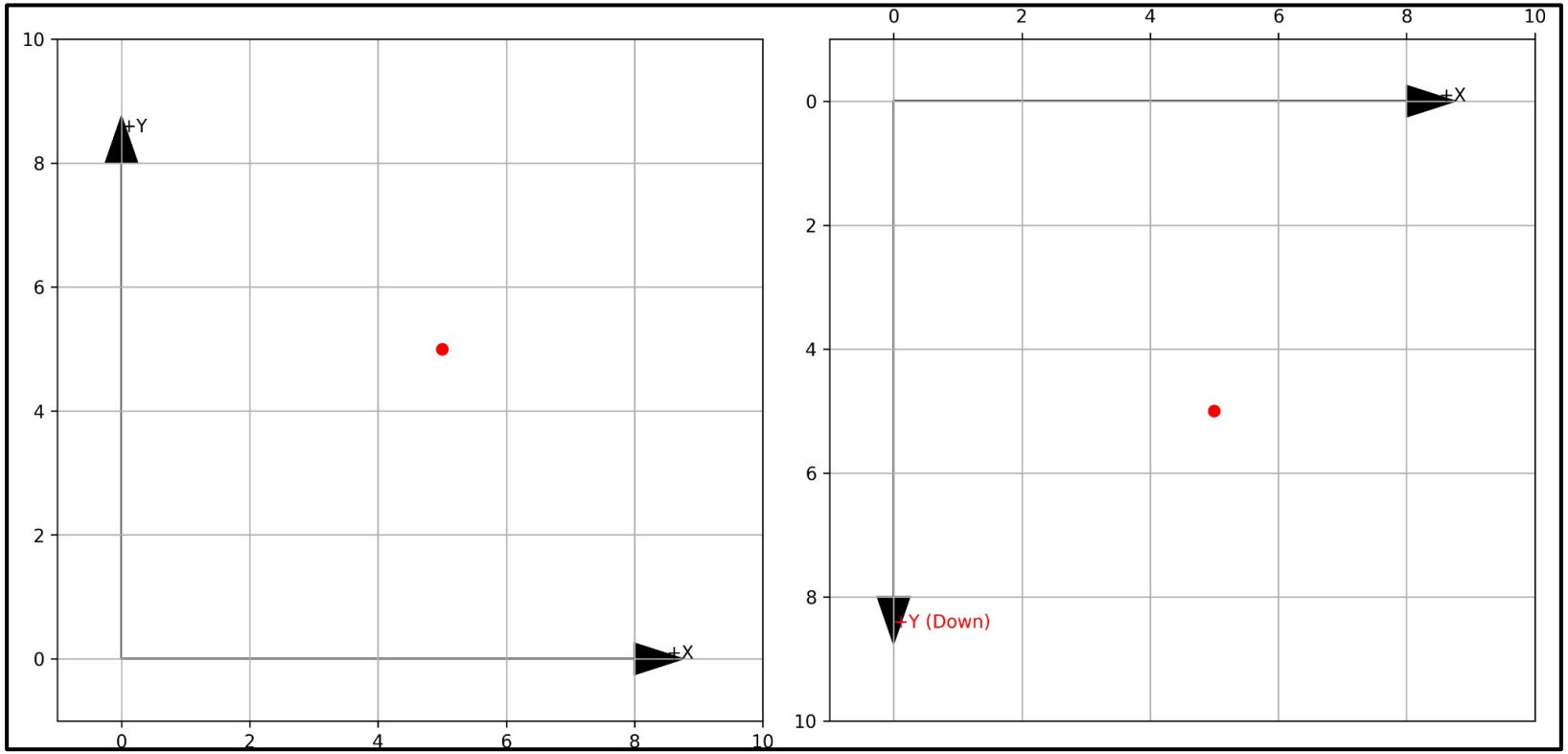
Geometry: Definition

- **We define the data in Python, but we must send it to the GPU to draw it**
  - We tell the GPU how to interpret our list of vertices using Primitives
    - GL_POINTS: Draw each vertex as a dot
    - GL_LINES: Connect pairs of vertices (0-1, 2-3)
    - GL_TRIANGLES: Connect triplets (0-1-2, 3-4-5)

Lecture Series Outline

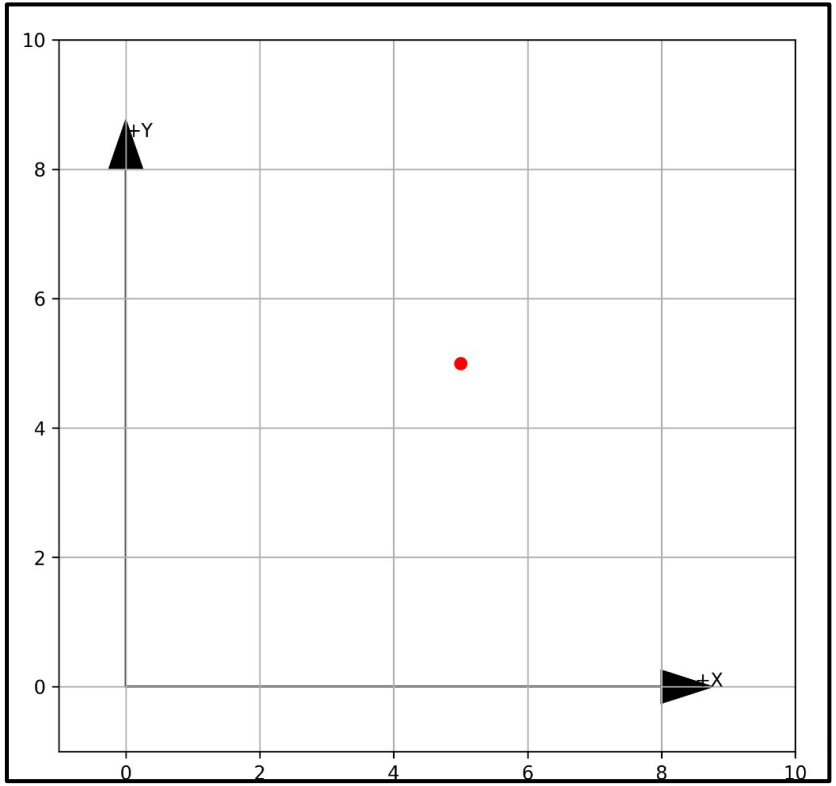
- **The Visual Computing Spectrum**
- **Deconstruction (Top-Down)**
- **Geometry (The Input)**
- **The Coordinate Systems**
  - World Space vs. Screen Space
- **The Image (The Output)**
- **The Bridge**

Coordinate Systems (1/4)



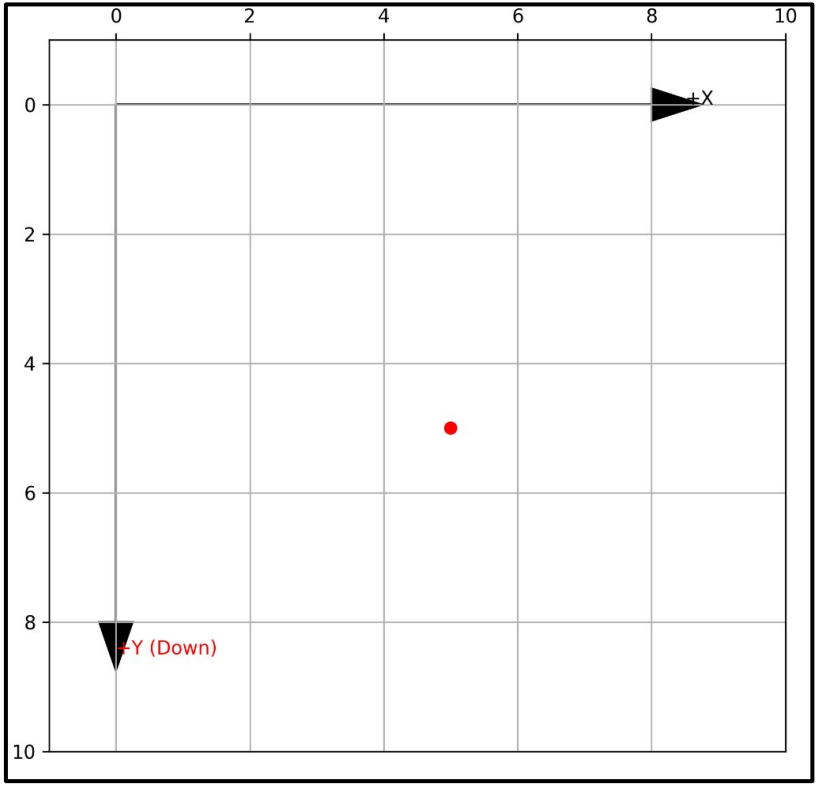
Coordinate Systems (2/4)

- **Data Mining uses Math Coordinates. Screens use Window Coordinates.**
  - They do not match
  - 1. Cartesian (World Space)
    - Origin (0,0) is Center
    - +Y is UP
    - Used for: Physics, 3D Models



Coordinate Systems (3/4)

- **Data Mining uses Math Coordinates. Screens use Window Coordinates.**
  - They do not match
  - 2. Screen Space (Window)
    - Origin (0,0) is Top-Left
    - +Y is DOWN
    - Used for: Pixels, Mouse Input



Coordinate Systems (4/4)

- **To solve the mismatch, OpenGL uses a "Standard Box"**
  - Normalized Device Coordinates (NDC)
  - NDC is a mathematical cube ranging from -1.0 to 1.0
    - Center: (0, 0)
    - Left Edge: x = -1
    - Right Edge: x = 1
    - Top Edge: y = 1
    - Bottom Edge: y = -1
  - Goal: We transform our game world into this box, and OpenGL flattens it to the screen

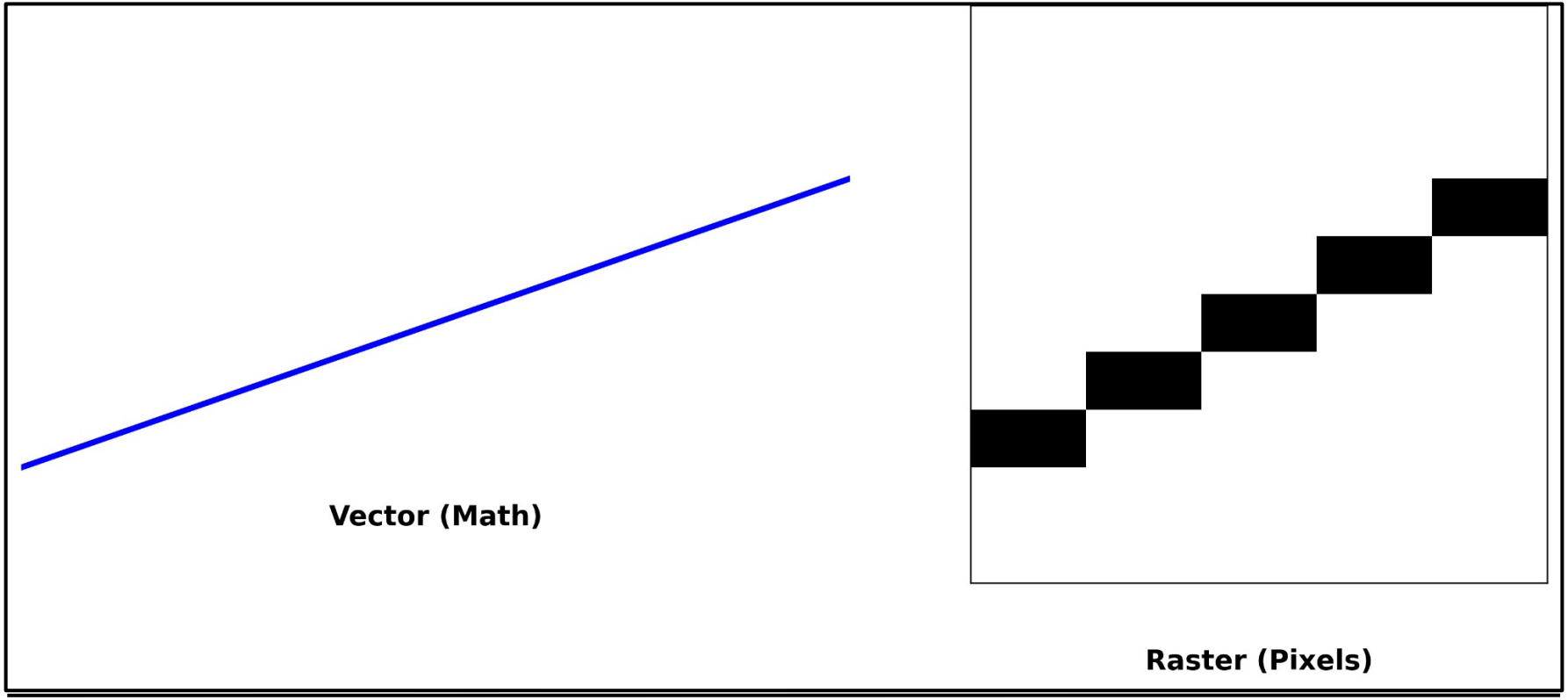
Lecture Series Outline

- **The Visual Computing Spectrum**
- **Deconstruction (Top-Down)**
- **Geometry (The Input)**
- **The Image (The Output)**
  - Pixels, Bit Depth, and Color Theory
  - Code: Configuring the Framebuffer
- **The Bridge**

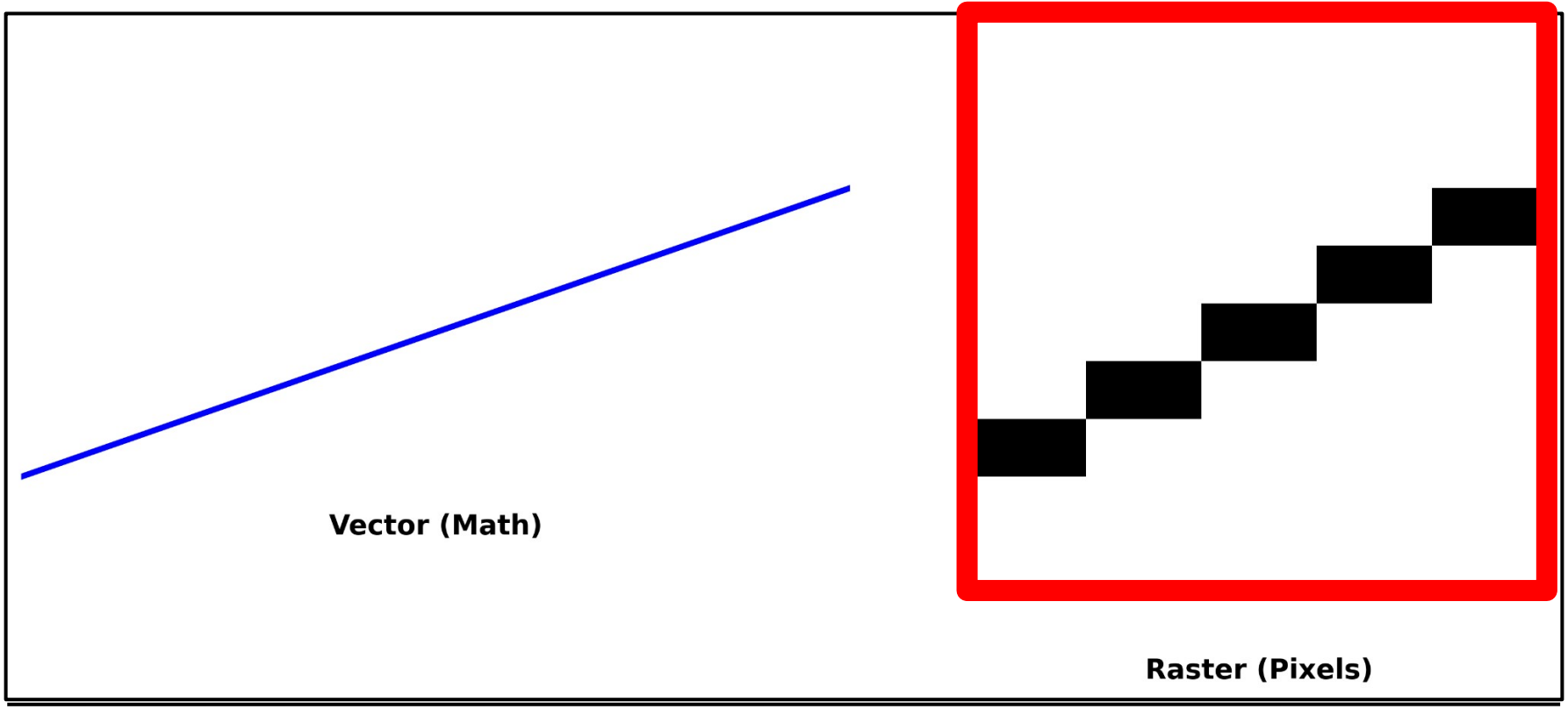
The Pixel (The Output): Paradigms: Raster vs. Vector (1/4)

- **We have defined our Mesh (Vector/Math)**
  - Now we need to display it (Raster)
  - Vector: Defined by equations
    - Infinite precision
  - Raster: Defined by a grid
    - Discrete precision

The Pixel (The Output): Paradigms: Raster vs. Vector (2/4)



The Pixel (The Output): Paradigms: Raster vs. Vector (2/4)



The Pixel (The Output): The Resolution (1/4)

- **Before we look at a single pixel, we must define the Grid**
  - Case Study: Our Polyfield screenshot
  - Dimensions: 1365 \times 612 pixels
  - Columns (Width): 1,365 discrete samples from left to right
  - Rows (Height): 612 discrete samples from top to bottom
  - Total Count: 835,380 pixels per frame

The Pixel (The Output): The Resolution (2/4)

- **Before we look at a single pixel, we must define the Grid**
  - Implication: To render this game at 60 FPS, the GPU must calculate the color for 50.1 Million pixels every second

The Pixel (The Output): The Resolution (3/4)

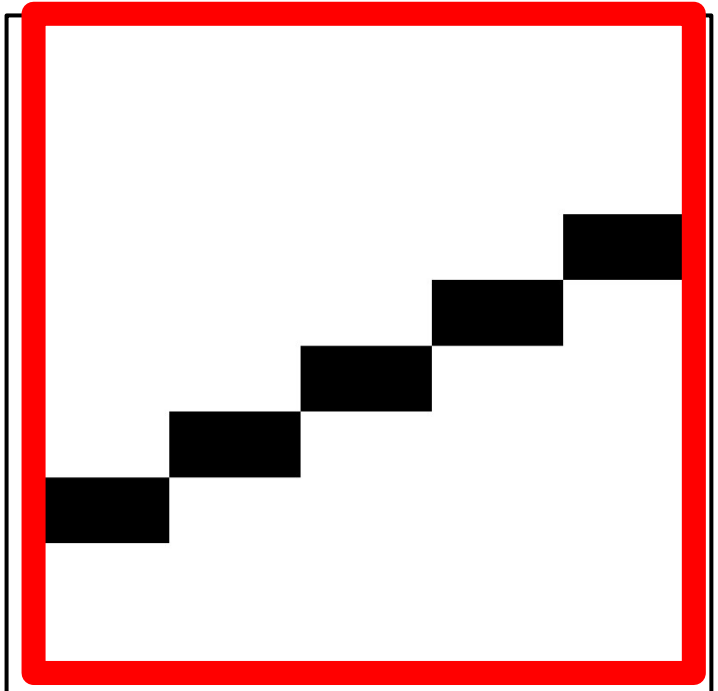
- **Before we look at a single pixel, we must define the Grid**
  - 1. Aspect Ratio (Shape)
    - Ratio of Width to Height: 1365 \div 612 \approx 2.23
    - This roughly aligns with 20:9, a common ultra-wide aspect ratio for modern Android phones
  - 2. Memory Cost (The Framebuffer)
    - If each pixel needs 3 Bytes (RGB), how big is one frame?

The Pixel (The Output): The Resolution (4/4)

- **Before we look at a single pixel, we must define the Grid**
  - 2. Memory Cost (The Framebuffer)
    - If each pixel needs 3 Bytes (RGB), how big is one frame?

The Pixel (The Output): Paradigms: Raster vs. Vector (1/2)

- **The Pixel is the smallest addressable unit on a screen**
  - Misconception: A pixel is not a little square. It is a Point Sample
  - Resolution: The dimensions of the grid (e.g., 1920 x 1080)
  - Quantization: Because pixels are discrete, diagonal lines look like stairs. This is called Aliasing (Jaggies)



Raster (Pixels)

The Pixel (The Output): Paradigms: Raster vs. Vector (2/2)

- In Python, we must ask the OS for a grid of pixels

```
import pygame as pg
# Request an 800x600 pixel grid
display_size = (800, 600)

# OPENGL: We will draw with 3D commands
# DOUBLEBUF: We use two screens (Front/Back) to prevent flickering
screen = pg.display.set_mode(display_size, pg.OPENGL | pg.DOUBLEBUF)
```

The Pixel (The Output): Bit Depth: 1-Bit (Binary)

- **How is a pixel stored in memory?**
  - Let's convert our Polyfield scene to 1-bit Color
    - Data: Each pixel is 0 (Black) or 1 (White)
    - Result: We lose texture and lighting. We only see high-contrast silhouettes
    - Context: Early LCD games or thermal printer outputs

The Pixel (The Output): Bit Depth: 8-Bit (Grayscale)

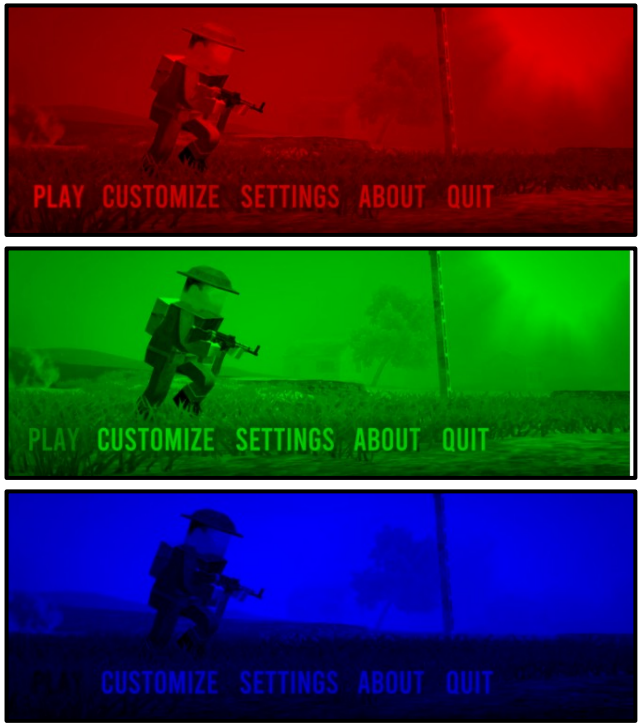
- **Now let's give each pixel 1 Byte (8 bits)**
  - Range: 0 (Black) to 255 (White)
  - Result: We can see the sun, the shadows on the grass, and the folds in the uniform
  - Limitation: We have Intensity, but no Color
  - Context: Medical Imaging (X-Rays)

The Pixel (The Output): Color Theory: The RGB Model (1/3)

- **Screens use Additive Color**
  - We mix light, not paint
  - We stack Three 8-bit channels together
    - 1. Red Channel: Intensity of Red (0-255)
    - 2. Green Channel: Intensity of Green (0-255)
    - 3. Blue Channel: Intensity of Blue (0-255)
    - Total Bits: 24 bits per pixel

The Pixel (The Output): Color Theory: The RGB Model (2/3)

- **Let's split the Polyfield scene into its components**
  - Red Channel: Strong in the uniform (Brown = Red + Green)
  - Green Channel: Strong in the grass
  - Blue Channel: Strong in the sky



The Pixel (The Output): Color Theory: The RGB Model (3/3)

- Crucial distinction when defining Color in Code (Float vs Int)
  - Pygame (CPU): Uses Integers (0, 255)
  - OpenGL (GPU): Uses Floats (0.0, 1.0)

```
# Integers (Standard Image Processing)
RED_INT = (255, 0, 0)
GREY_INT = (128, 128, 128)

# Floats (Graphics Card)
# We normalize: 128 / 255 = 0.5
glColor3f(0.5, 0.5, 0.5)
```

The Pixel (The Output): Color Spaces: sRGB vs Linear

- **The Problem: Monitors are non-linear. They make dark colors too dark**
  - sRGB: The standard color space for Web and Monitors
  - Gamma Correction
    - The GPU calculates math in Linear Space (Physics)
    - We must convert to sRGB before displaying so the human eye sees it correctly
    - If your game looks "washed out" or "too dark", it is usually a Gamma issue

The Pixel (The Output): Color Spaces: The Alpha Channel (RGBA)

- **Modern Graphics uses 32-bit Color**
  - The extra 8 bits are for Alpha (A)
  - Definition: Opacity
    - A = 0.0: Transparent
    - A = 1.0: Opaque
  - Usage: The Polyfield HUD (Health bar, Ammo count) uses Alpha to float over the 3D world

Q & A Session

- Comments, concerns and complaints?

Bibliography

1. **2025/26 CSC 4505**
   https://moodle.unza.zm/enrol/index.php?id=624

csc4505@unza.zm

https://moodle.unza.zm/course/view.php?id=626

YouTube Playlist ID: PLubb3AIPdPhYGt3ax8orjiDo3cuK0tGfx

CSC 4505 (2025/26)
Graphics and Visual Computing

Lecture 2: Fundamentals of Visual Computing

Lighton Phiri <lighton.phiri@unza.zm>
Department of Computing and Informatics
University of Zambia
https://datalab.unza.zm/people/lightonphiri